

WSpeedrun API Service

Software Architecture Lab Documentation



Kelompok 5:

Jason Nathaniel Prijatno - 2802390136

Naufal Maulana Ichlas - 2802391416

Yupriando - 2802392204

Architecture: Microservices (NestJS, Prisma ORM, MySQL)

UNIVERSITAS BINA NUSANTARA
TANGERANG, INDONESIA

2026

1. Deskripsi Proyek & Arsitektur Sistem

WSpeedrun API merupakan sistem *backend* terpusat yang dirancang untuk mencatat, mengelola, dan memvalidasi rekor waktu penyelesaian (*speedrun*) dari berbagai judul game kompetitif. Proyek ini dibangun dengan menerapkan prinsip dan pola arsitektur **Microservices** guna memastikan skalabilitas, kemudahan pemeliharaan, serta pemisahan tanggung jawab (*separation of concerns*) yang optimal di tingkat sistem.

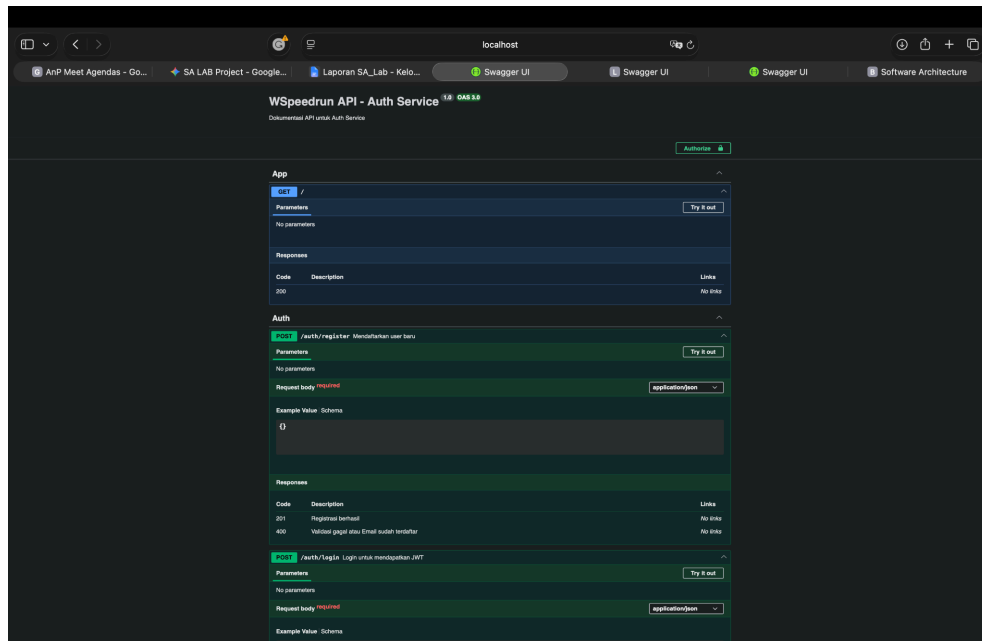
Keseluruhan domain logika didistribusikan ke dalam tiga layanan (*services*) yang beroperasi secara independen pada *port* jaringan yang berbeda. Meskipun terisolasi di tingkat aplikasi, ketiga ekosistem layanan ini tetap terintegrasi secara terstruktur melalui pendekatan *shared database* menggunakan MySQL dan pengelolaan skema relasional via Prisma ORM.

Berikut adalah pemetaan operasional dari masing-masing layanan:

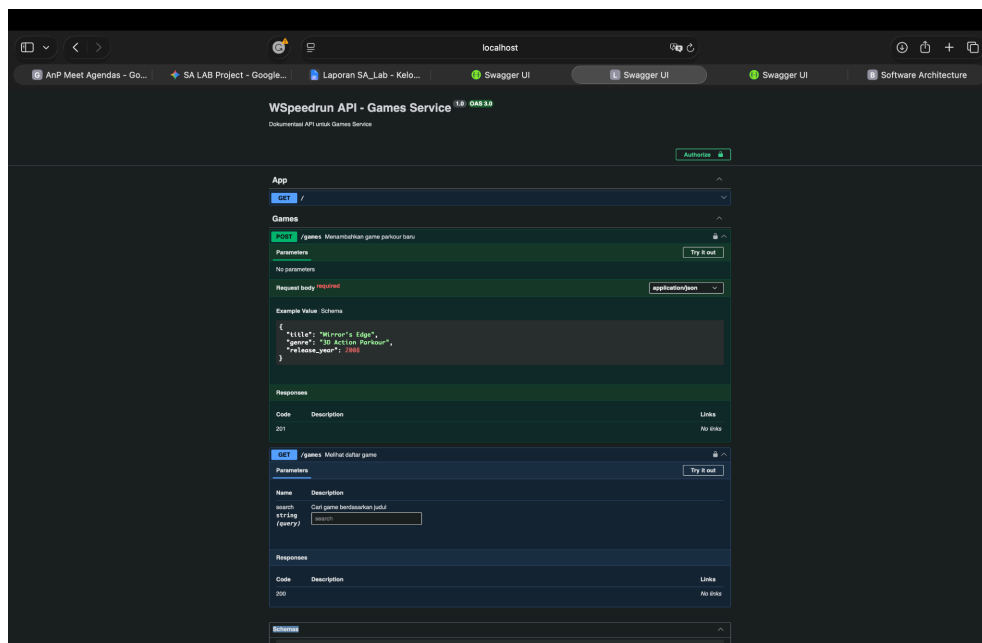
- **Auth Service (Port 3000):** Layanan khusus untuk menangani otentikasi pengguna, registrasi, enkripsi *password* menggunakan Bcrypt, dan otorisasi keamanan berbasis *stateless* token (Passport JWT).
- **Games Service (Port 3001):** Layanan terpusat untuk mengelola master data dan entitas katalog game kompetitif.
- **Runs Service (Port 3002):** Layanan inti yang menangani pencatatan rekor waktu (*speedrun*), validasi lintas domain, serta menghasilkan luaran papan klasemen (*Leaderboard*) yang dinamis. Layanan ini menerapkan arsitektur *microservices* murni dengan melakukan validasi *Service Calling* menggunakan pustaka **Axios**. Sebelum menyimpan entitas rekor baru ke dalam *database*, Runs Service secara asinkron memverifikasi eksistensi `user_id` ke Auth Service (Port 3000) dan `game_id` ke Games Service (Port 3001), memastikan integritas relasional data tetap terjaga meski berada di ekosistem layanan yang terisolasi.

2. Dokumentasi API (Swagger UI)

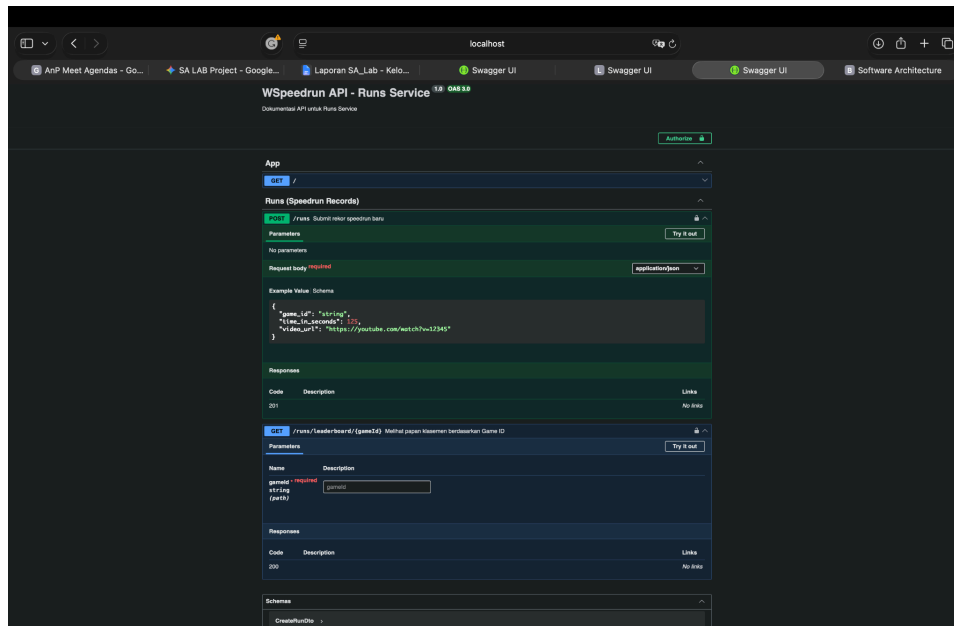
Seluruh rute API dapat diakses dan diuji coba melalui Swagger OpenAPI yang terisolasi di masing-masing layanan.



Gambar 2.1 - Antarmuka Swagger untuk Auth Service.



Gambar 2.2 - Antarmuka Swagger untuk Games Service.



Gambar 2.3 - Antarmuka Swagger untuk Runs Service.

3. Panduan Instalasi (Developer Guide)

Panduan ini ditujukan bagi *developer* atau penguji untuk merestorasi dan menjalankan ketiga *microservices* ini di *local machine*.

A. Persiapan Database

1. Pastikan modul **MySQL** pada XAMPP (atau *database server* lokal lainnya) dalam keadaan aktif (*Running* di port 3306).
2. Buka <http://localhost/phpmyadmin> di *browser*.
3. Buat *database* baru dengan nama persis: **wspeedrun_db**.
4. Lakukan *Import* file **wspeedrun_db.sql** (yang terlampir di dalam folder pengumpulan tugas) ke dalam *database* tersebut.

B. Instalasi Dependencies (Node.js)

Karena folder **node_modules** telah dihapus untuk mengoptimalkan ukuran *file*, Kita perlu mengunduh ulang *package* yang dibutuhkan. Buka terminal dan jalankan perintah ini pada masing-masing folder (**wspeedrun-auth**, **wspeedrun-games**, dan **wspeedrun-runs**):

Bash

npm install

C. Konfigurasi Environment & Prisma ORM

Setiap *service* memerlukan koneksi ke *database*. Lakukan langkah ini secara berulang di ketiga folder *service*:

1. Buat *file* baru bernama **.env** di *root* folder masing-masing proyek (sejajar dengan **package.json**).
2. Masukkan parameter *database connection string* berikut ke dalam file **.env**:
DATABASE_URL="mysql://root:@localhost:3306/wspeerdrun_db" (*Sesuaikan kredensial root dan password jika MySQL Anda menggunakan pengaturan berbeda*).
3. Buka terminal di masing-masing folder dan *generate* Prisma Client dengan perintah:

```
npx prisma generate
```

D. Menjalankan Server

Untuk menjalankan ekosistem secara utuh, buka tiga tab terminal yang berbeda (satu untuk setiap folder), lalu jalankan perintah penjalan *server* secara bersamaan:

```
npm run start:dev
```

Sistem akan menyala secara paralel, dan API siap diuji coba melalui:

- **http://localhost:3000/api**
- **http://localhost:3001/api**
- **http://localhost:3002/api**

E. JWT Token:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjQ0NmFiNDc2LWE1NDctND
```

dhNi1hMjJlLWMwYjZhOThhNmRkZCIsInJvbGUiOiJVU0VSIiwiaWF0IjoxNzgwNDc5N
DU2LCJleHAiOjE3ODAxNjU4NTZ9.JgUeaWtB08IN8PlU6au9wCS1qgFDinbQ-eiv5dI
_kco

F. Skenario Pengujian Integrasi Microservices (Service Calling)

Sistem ini telah dirancang untuk menolak data yatim (*orphan data*). Untuk membuktikan bahwa komunikasi antar-layanan (*Service Calling*) via Axios berjalan dengan sempurna, penguji dapat menjalankan skenario validasi silang berikut:

- 1. Pembuatan Entitas Induk:** Buka Swagger Auth Service (Port 3000) dan lakukan registrasi pengguna baru via *endpoint POST /auth/register*. Selanjutnya, buka Swagger Games Service (Port 3001) dan daftarkan game baru via *POST /games*. Salin nilai *id* dari kedua respons tersebut.
- 2. Pengujian Positif (Valid Data):** Buka Swagger Runs Service (Port 3002) dan catat rekor baru via *POST /runs*. Masukkan kombinasi *user_id* dan *game_id* yang valid dari langkah 1. Sistem akan mengembalikan HTTP Status *201 Created*, membuktikan bahwa Runs Service berhasil menelepon dan memvalidasi kebenaran ID ke dua layanan lainnya sebelum menyimpan data ke *database*.
- 3. Pengujian Negatif (Invalid Data):** Lakukan *request* yang sama pada *POST /runs*, namun ubah nilai *user_id* atau *game_id* menjadi ID fiktif yang tidak pernah terdaftar. Sistem akan memblokir proses penyimpanan dan mengembalikan HTTP Status *404 Not Found* (contoh: "*Validasi Microservice Gagal: User tidak ditemukan di Auth Service*"), membuktikan bahwa isolasi dan validasi lintas *port* beroperasi dengan presisi absolut.